

UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BTECH CSE – CS 201

SEMESTER: 6TH

CLOUD COMPUTING

(23CSH-307)

Unit No. 2

Chapter No. 2

Lecture No. 17

Topic : Service registry patterns, client- side vs. server-side discovery,
role of API Gateway in microservices (routing, security,
throttling).

Sanjay Tiwari, E18823

Senior Technical Trainer

Recap

- **Application Packaging:** Containers provide lightweight and efficient packaging by bundling applications with their dependencies for consistent execution.
- **The Orchestration Imperative:** While Docker standardizes container usage, orchestration is necessary when managing containers at scale.
- **Platform Trade-offs:** Kubernetes offers powerful orchestration features, whereas ECS provides a simplified approach to container management.
- **System Reliability:** These technologies support distributed architectures and ensure the reliability, scalability, and availability of cloud-native systems.

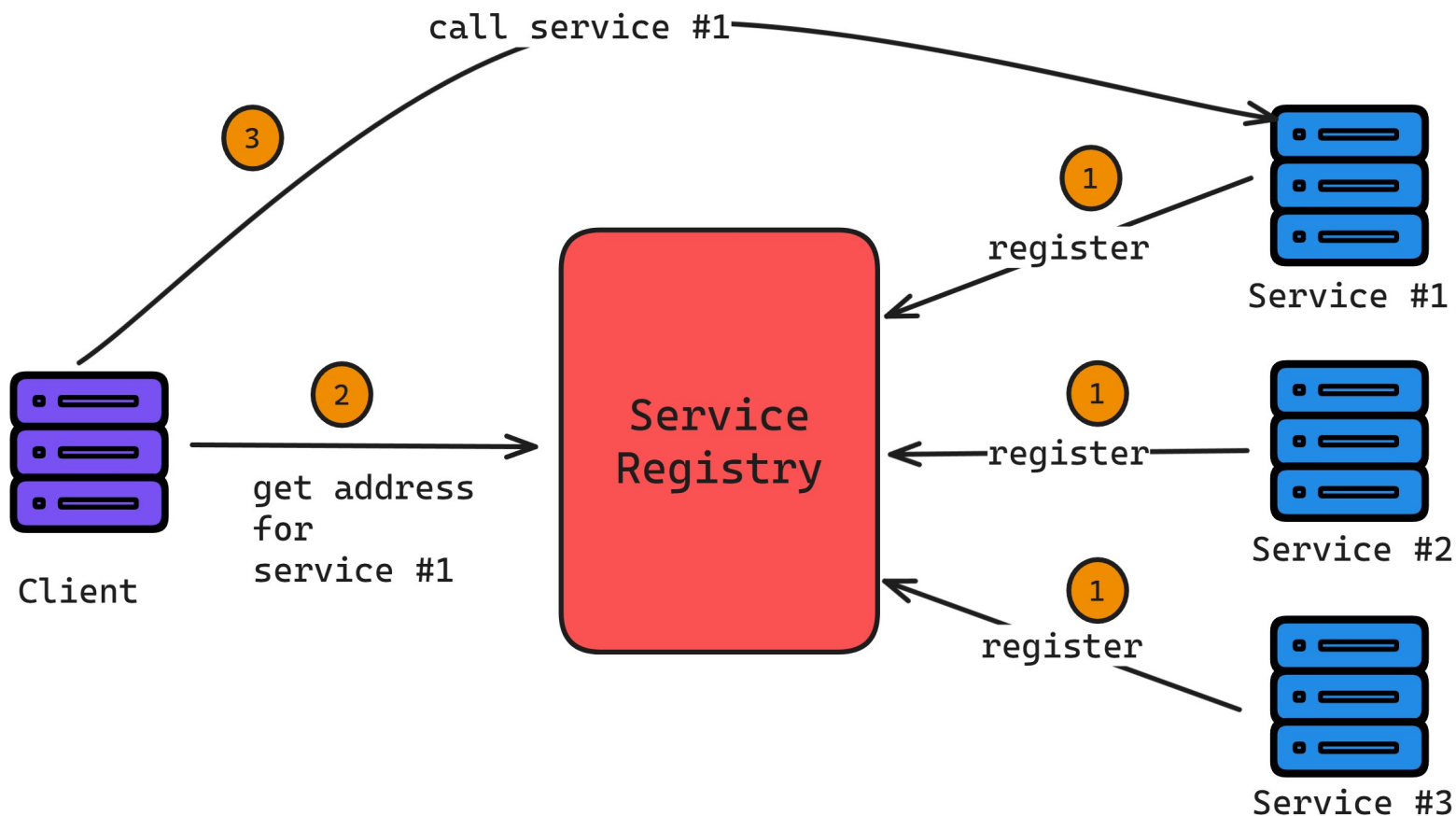
Learning Objectives

- Service Registry Concepts
- Service Discovery Mechanisms
- Client-Side vs Server-Side Discovery
- Role of API Gateway

Service Registry Overview

- **The Central Directory:** A service registry acts as a central directory that stores the names and locations of all service instances.
- **Dynamic Adaptation:** It handles dynamic IP and port changes, making it essential in dynamic environments where services scale frequently.
- **Health and State:** The registry supports service registration, handles deregistration, and actively keeps track of healthy services.
- **Architectural Decoupling:** By centralizing service metadata, it reduces hard-coded dependencies and enables loose coupling across the system.

Service Registry Overview



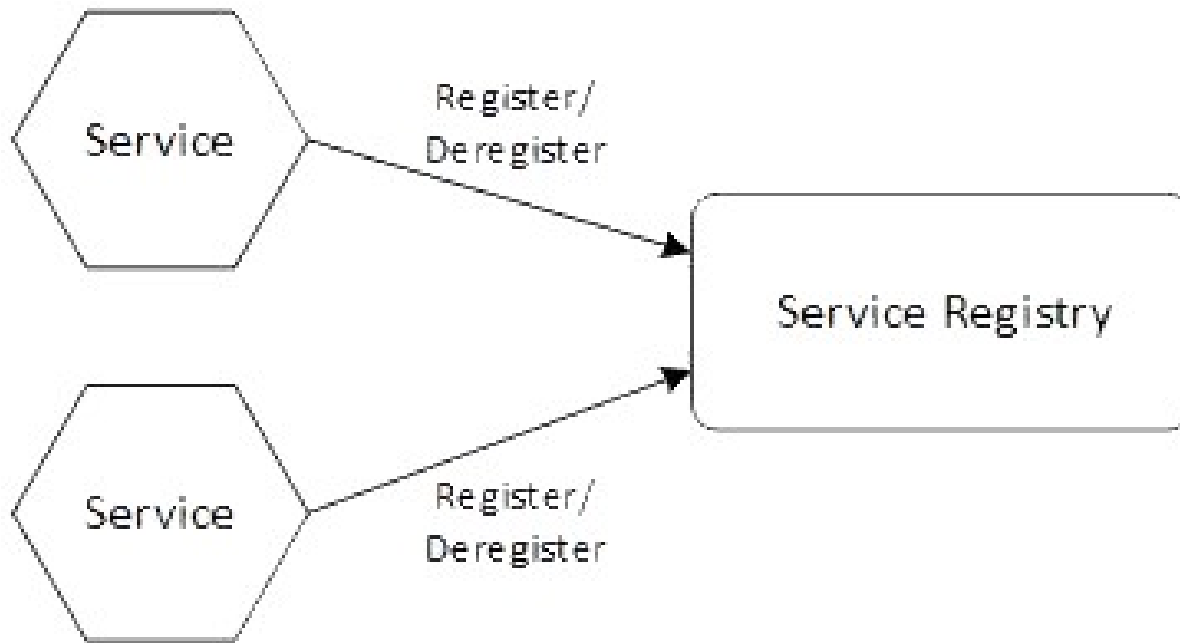
ScalableThread.com

Source: Scalable Thread

Service Registration Patterns

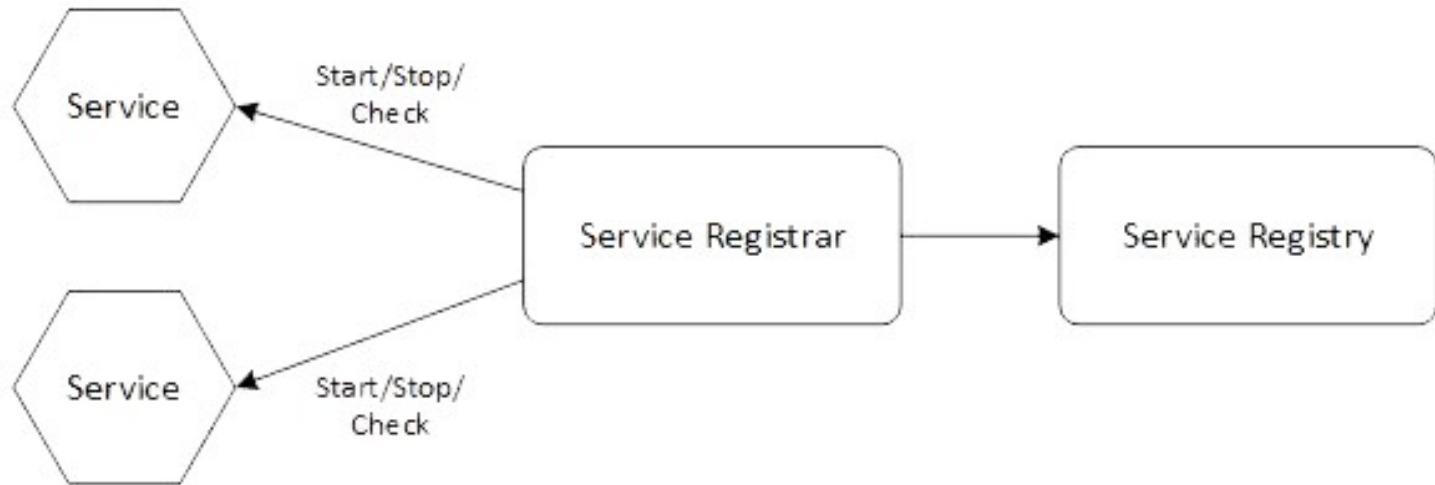
- **Self-Registration Pattern:** A service provides its own metadata and registers itself at startup. It sends periodic health updates and deregisters on shutdown. This requires service-side logic but directly reflects the service state.
- **Third-Party Registration Pattern:** An external component handles registration centrally, and services remain completely unaware of the registry. Health checks are performed externally. This is useful when services cannot modify code, though it requires additional infrastructure.

Self Registration Pattern



Source: O'Reilly

Third-Party Registration Pattern



Source: O'Reilly

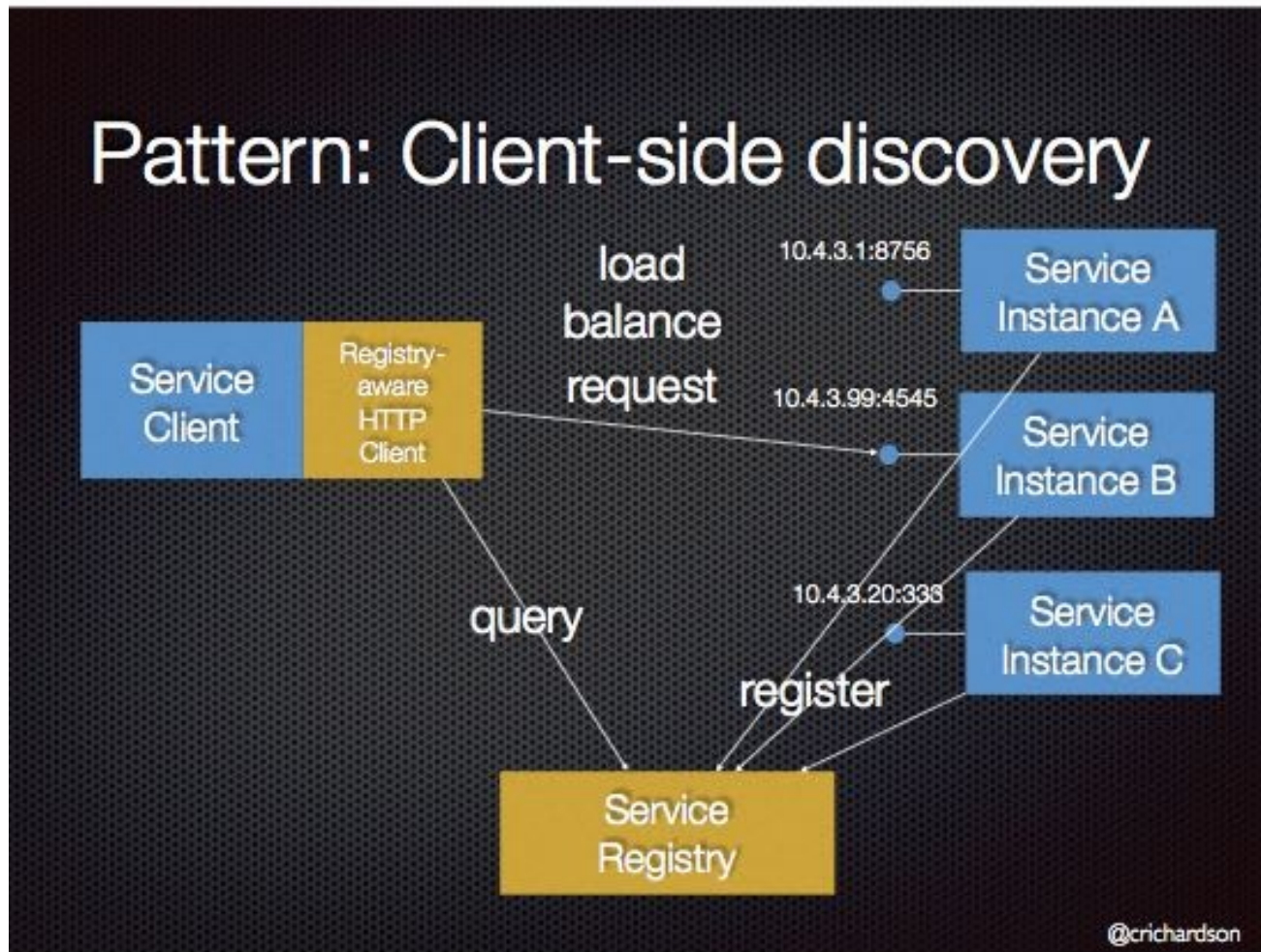
Service Discovery Mechanisms

- **The Discovery Process:** Service discovery is the fundamental process of locating service instances utilizing the registry as an information source.
- **Dynamic Communication:** By avoiding fixed endpoints, it enables fault-tolerant inter-service communication even in highly dynamic scaling scenarios.

Client-Side Service Discovery

- **Direct Interaction:** The client queries the service registry directly and selects the appropriate service instance.
- **Client Autonomy:** Load balancing logic resides within the client, allowing for direct communication with the backend service.
- **Trade-offs:** This approach provides more control and flexibility for the client. However, it requires client-side intelligence and custom libraries, which increases client complexity and system coupling.

Client-Side Service Discovery

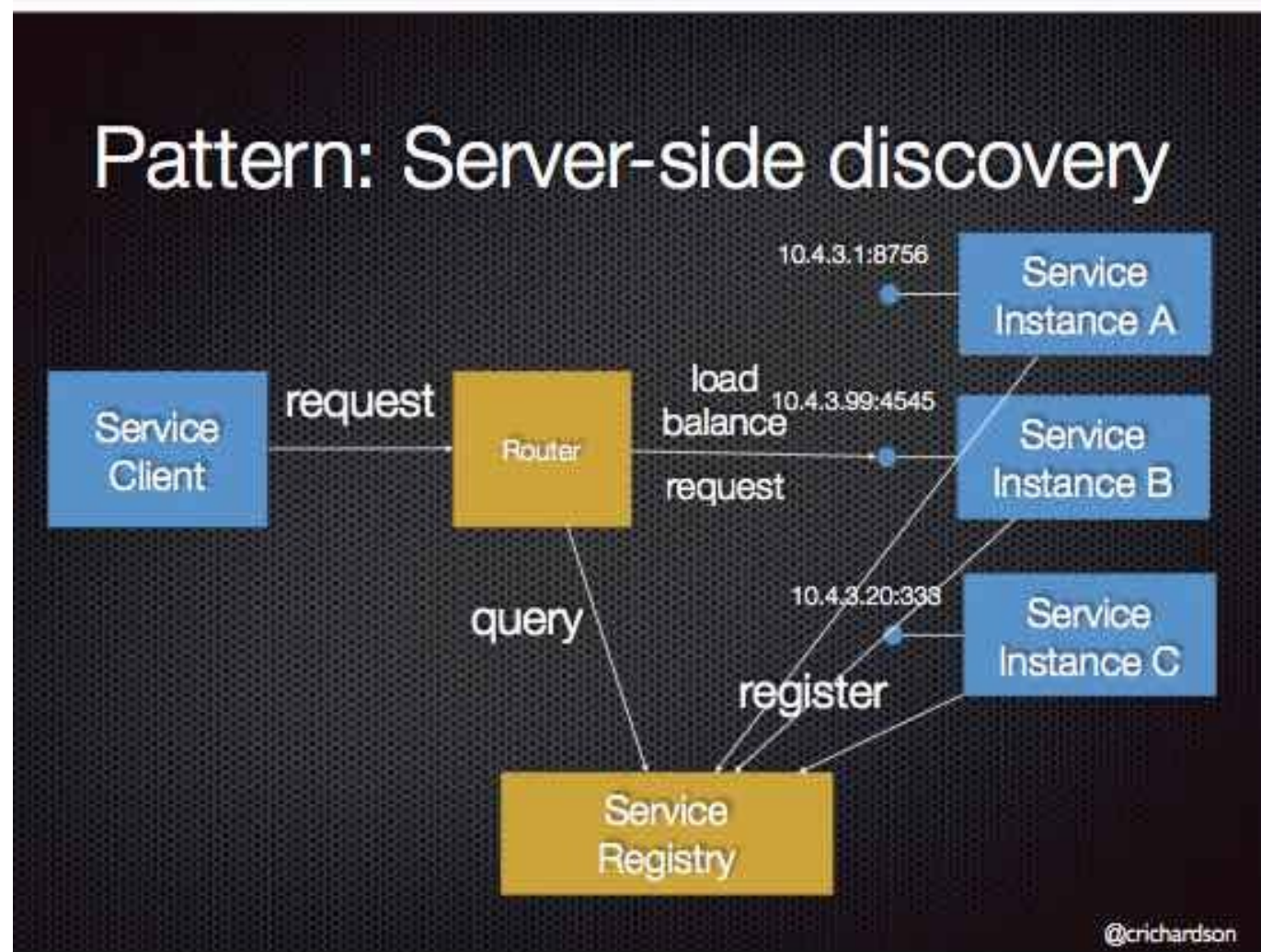


Source: Microservices.io

Server-Side Service Discovery

- **Intermediary Routing:** The client sends a request to an intermediary, which then queries the registry and handles routing externally.
- **Client Simplicity:** Because the client is unaware of service locations, this heavily simplifies client implementation and logic.
- **Trade-offs:** It provides centralized load balancing and centralizes routing logic, making changes easier to manage. However, it requires additional infrastructure to operate.

Server-Side Service Discovery



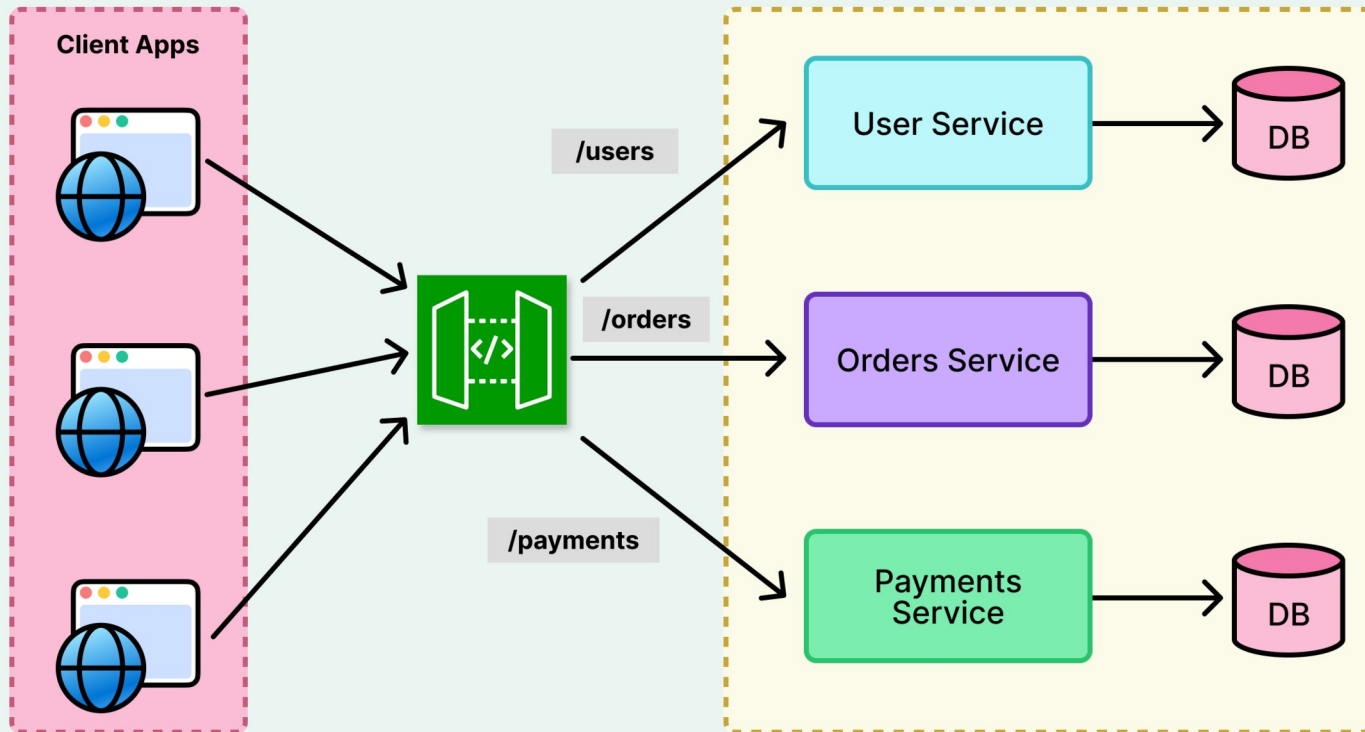
Source: Microservices.io

Introduction to API Gateways

- **The Single Entry Point:** An API Gateway acts as a single entry point for clients, effectively routing requests to the proper backend services.
- **Topological Abstraction:** It abstracts the internal service structure and hides the internal service topology from the outside world.
- **Security & Simplicity:** This pattern reduces direct service exposure, centralizes cross-cutting concerns, and significantly simplifies client interaction.

API Gateways

API Gateway for Microservice Architecture



Source: ByteByteGo

API Gateway Capabilities: Routing

- **Traffic Direction:** The gateway intelligently directs requests to the correct service.
- **Advanced Routing Protocols:** It supports advanced techniques like path-based routing and version-based routing.
- **Network Optimization:** It enables request aggregation to combine multiple internal calls into one, which simplifies client communication.
- **Adaptability:** The gateway supports dynamic service changes and significantly improves request management.

API Gateway Capabilities: Security & Throttling

- **Access Control:** The gateway acts as a security shield, handling authentication and authorization while enforcing access control policies.
- **Backend Protection:** It protects backend services by preventing abuse and overload.
- **Traffic Management:** It limits request rates (throttling), enables API key management, and supports traffic shaping.
- **System Integrity:** Collectively, these measures drastically improve overall system stability.

Summary of the Lecture

6

- **Dynamic Environments:** Service registries enable dynamic discovery, which fundamentally removes hard-coded dependencies and supports scalability.
- **Discovery Trade-offs:** Client-side and server-side discovery offer distinct architectural trade-offs regarding complexity and flexibility.
- **Centralized Access:** The API Gateway centralizes access, utilizing advanced routing to simplify client interaction.
- **System Reliability:** Implementing security and throttling at the gateway protects internal services, making these patterns essential for scalable microservices.



Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other¹⁰ relevant learning resources

- Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview
- Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

Next Lecture

Introduction to serverless (FaaS), benefits (no ops, cost efficiency), limitations (cold starts, execution limits), use cases

Quiz/ FAQ's

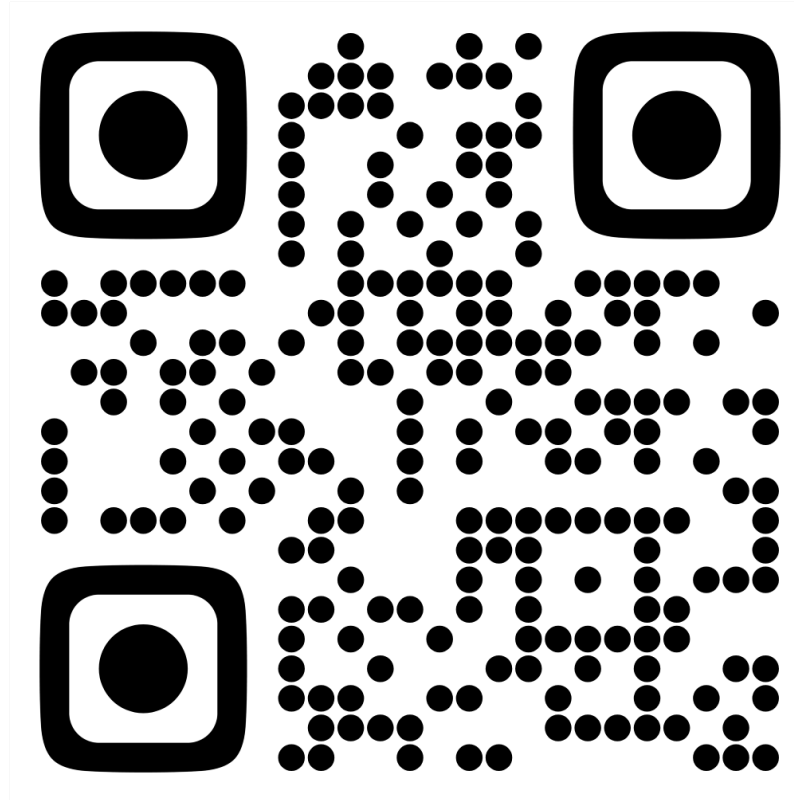
1. What is a service registry in microservices?
2. What is the difference between client-side and server-side service discovery?
3. Why is service discovery important in microservices?
4. What is the main role of an API Gateway?
5. Why is throttling implemented at the API Gateway?

References/ Articles/ Videos

- **Martin Fowler – Service Registry and Discovery**
<https://martinfowler.com/articles/microservices.html>
- **Microsoft – API Gateway Pattern**
<https://learn.microsoft.com/en-us/azure/architecture/patterns/api-gateway>
- **AWS – Service Discovery Overview**
<https://docs.aws.amazon.com/cloud-map/latest/dg/what-is-cloud-map.html>
- **Red Hat – API Gateway in Microservices**
<https://www.redhat.com/en/topics/microservices/what-is-an-api-gateway>

Class-Wise Feedback

11



Thank You

For queries

Email: sanjay.e18823@cumail.in